



Nexora — Mini E-commerce Distribuído Relatório

Aluno: Caio Ferreira Lira de Oliveira | Professor: Jorge

1. Como a comunicação entre os microsserviços foi implementada?

- Via HTTP/REST síncrono usando RestTemplate. O API Gateway (porta 5000) recebe as requisições e encaminha via ProxyService, repassando o JWT no header Authorization. O Serviço de Pedidos consulta o Serviço de Produtos via HTTP (URL via variável de ambiente PRODUCTS_URL) para validar estoque. Cada serviço roda em porta separada (5001, 5002, 5012, 5003), de forma independente, sempre comunicando-se por rede.

2. Qual estratégia de consistência foi adotada na replicação? Por quê?

- Consistência forte com política 2-of-2 write: toda escrita precisa ser confirmada em ambas as réplicas antes de retornar sucesso. Isso evita inconsistências críticas no catálogo (ex.: produto visível só em uma réplica). O ReplicationService verifica saúde da réplica, salva localmente, propaga via POST /internal/products/replicate e só retorna 201 após confirmação das duas. Trade-off: maior latência nas escritas e leituras usam round-robin entre réplicas.

3. O que acontece com o sistema se o Serviço de Pedidos cair?

- O sistema continua operando com funcionalidade degradada, Usuários e Produtos permanecem funcionais e apenas pedidos ficam indisponíveis. O HeartbeatScheduler do Gateway faz GET /health a cada 5s e após 2 falhas consecutivas, o ServiceRegistry marca o serviço como DOWN e registra o evento. Requisições a /orders/* recebem 503 Service Unavailable. Quando o serviço volta, o próximo heartbeat bem-sucedido restaura o status para UP automaticamente, sem intervenção manual.

4. Como o JWT garante que um usuário comum não consiga criar produtos?

- O JWT contém o campo role no payload, assinado com HMAC-SHA256 com chave secreta compartilhada. No login, o UsersService gera o token com userId, email, role e exp. O Gateway repassa o token para o ProductsService, que valida via JwtService.validateToken(). Se role não for "admin", retorna 403 Forbidden. Qualquer alteração no payload invalida a assinatura, impedindo forjar o token.

5. Quais limitações a implementação possui em relação a um sistema real de produção?

- Persistência em JSON: a ausência de transações ACID gera risco de corrupção de dados em cenários de escrita concorrente. Em produção, isso seria resolvido com bancos de dados como PostgreSQL ou MongoDB, que oferecem suporte nativo a transações.
- JWT sem mecanismo de revogação: caso um token seja roubado, ele permanece válido até sua expiração natural (24h). A solução seria implementar uma blacklist em Redis ou adotar token rotation.
- Ausência de circuit breaker: falhas em um serviço podem se propagar em cascata, bloqueando threads de serviços dependentes. Bibliotecas como Resilience4j ou Hystrix resolveriam esse problema.
- Replicação síncrona simples: o modelo atual não possui quorum nem leader election, o que limitaria a escalabilidade e a tolerância a falhas em clusters maiores. Algoritmos como Raft/Paxos, ou o uso de Kafka, seriam alternativas mais robustas.
- Gateway sem autoscaling: o gateway atua como ponto único de falha (single point of failure). Em produção, seria necessário orquestrá-lo com Kubernetes, mantendo múltiplas réplicas.

- Ausência de TLS interno: a comunicação entre microsserviços ocorre em texto claro, expondo dados sensíveis na rede interna. A solução seria adotar mTLS com certificados internos entre os serviços.
- Heartbeat simplificado: o mecanismo atual detecta apenas indisponibilidade total, não identificando degradação parcial (lentidão). Uma solução seria integrar Prometheus com métricas de latência para monitoramento mais granular.